

DATABASE, MODELLO CONCETTUALE, LOGICO ED ESEMPIO TRATTO DALLA MATURITA'

Sommario

Modello Concettuale e Logico di un Database Relazionale	2
1. Introduzione	2
2. Modello Concettuale	2
2.1 Entità.....	2
2.2 Attributi	2
2.3 Chiavi	2
2.4 Relazioni.....	2
3. Modello Logico	3
3.1 Regole di trasformazione.....	3
3.2 Caratteristiche del modello logico.....	3
4. Esempio Completo.....	3
Requisito:	3
Modello Concettuale (E-R)	3
Modello Logico (Relazionale)	4
5. Vincoli di dominio e vincoli referenziali.....	4
Vincoli di Dominio.....	4
Vincoli Referenziali	4
Da non fare: permettere ordini senza cliente (chiave esterna facoltativa)	5
In sintesi.....	5
6. Conclusione	5
Traccia della prova di maturità informatica	6
Esempio concreto (pratico) – Car pooling	6
1. Realtà descritta	6
2. Modello concettuale (schema E-R).....	6
3. Modello logico (schema relazionale).....	7
Come si lega al “modello concettuale e logico” richiesto dalla prova	7

Modello Concettuale e Logico di un Database Relazionale

1. Introduzione

Quando si progetta un database relazionale non si parte subito dalla creazione delle tabelle in SQL. Il processo si sviluppa in **più fasi**:

1. **Analisi dei requisiti** → capire cosa serve al sistema.
2. **Modello concettuale** → rappresentazione astratta dei dati (indipendente dal DBMS).
3. **Modello logico** → traduzione del concettuale in relazioni (tabelle).
4. **Modello fisico** → implementazione concreta nel DBMS scelto (MySQL, Oracle, SQL Server...).

2. Modello Concettuale

Strumento principale: **Diagramma E-R (Entity–Relationship)**.

È una rappresentazione grafica che descrive le **entità**, gli **attributi** e le **relazioni**.

2.1 Entità

- Oggetti di interesse del mondo reale.
- Esempio: **Studente**, **Corso**, **Docente**.
- Si rappresentano con **rettangoli**.

2.2 Attributi

- Proprietà delle entità o delle relazioni.
- Esempio: Studente ha attributi → *Matricola*, *Nome*, *Cognome*, *DataNascita*.
- Si rappresentano con **ovali**.
- Esistono attributi:
 - **Semplici** (Nome)
 - **Composti** (Indirizzo → Via, Città, CAP)
 - **Derivati** (Età, calcolata da DataNascita).

2.3 Chiavi

- Ogni entità deve avere una **chiave primaria** → identifica univocamente ogni istanza.
- Esempio: Matricola per Studente.

2.4 Relazioni

- Collegano le entità.
- Si rappresentano con **rombi**.
- Tipi di cardinalità:
 - **1:1** → uno studente ha una sola tessera mensa.
 - **1:N** → un docente tiene molti corsi, ma un corso ha un solo docente.
 - **N:M** → uno studente segue molti corsi e ogni corso ha molti studenti.

Il modello concettuale è indipendente dalla tecnologia: serve solo a capire la struttura dei dati.

3. Modello Logico

Il modello logico traduce il **diagramma E-R** in un insieme di **tabelle relazionali**.

3.1 Regole di trasformazione

1. **Ogni entità diventa una tabella.**
 - Gli attributi diventano colonne.
 - La chiave primaria rimane chiave della tabella.
2. **Relazioni 1:N** → si realizza aggiungendo una **chiave esterna (FK)** dal lato N.
 - Esempio:
 1. Qui *idCorso* è FK che collega Studente → Corso.
 - Tabella **Corso**(idCorso, NomeCorso, Docente)
 - Tabella **Studente**(Matricola, Nome, Cognome, idCorso)
3. **Relazioni N:M** → si realizza con una **tabella di collegamento**.
 - Esempio: Studente–Corso
 - **Studente**(Matricola, Nome, Cognome)
 - **Corso**(idCorso, NomeCorso)
 - **Iscrizione**(Matricola, idCorso, DataIscrizione)
 - PK di Iscrizione = (Matricola, idCorso).

3.2 Caratteristiche del modello logico

- Dipende dal modello relazionale.
- Non tiene conto ancora delle prestazioni (quelle appartengono al modello fisico).
- È già pronto per essere tradotto in SQL.

4. Esempio Completo

Requisito:

Gestire studenti e corsi universitari, con possibilità di iscrizione a più corsi.

Modello Concettuale (E-R)

- Entità: **Studente**, **Corso**
- Relazione: **Iscrizione** (N:M)

- Attributi:
 - Studente: Matricola (PK), Nome, Cognome
 - Corso: idCorso (PK), NomeCorso, Crediti
 - Iscrizione: DataIscrizione

Modello Logico (Relazionale)

- Studente(Matricola, Nome, Cognome)
- Corso(idCorso, NomeCorso, Crediti)
- Iscrizione(Matricola, idCorso, DataIscrizione)
 - FK (Matricola) → PK di Studente
 - FK (idCorso) → PK di Corso

5. Vincoli di dominio e vincoli referenziali

Vincoli di Dominio

* Definiscono ****l'insieme dei valori ammessi**** per un attributo (colonna) di una tabella.
 * Servono a garantire la ****correttezza e coerenza dei dati****, impedendo l'inserimento di valori non validi.

Esempi:

- * Tipo di dato: `DataNascita` deve essere di tipo `DATE`.
- * Range: `Età` deve essere compresa tra 0 e 120.
- * Formato: `Email` deve rispettare la sintassi di un indirizzo valido.
- * Enumerazione: `StatoCivile` ∈ {`Celibe`, `Sposato`, `Divorziato`}.

In SQL, un vincolo di dominio si implementa con:

- * ****CHECK****: `CHECK (Età >= 0 AND Età <= 120)`
- * ****DEFAULT****: valore predefinito (es. `DEFAULT 'Italia'`)
- * ****NOT NULL****: impedisce valori nulli.

Vincoli Referenziali

- * Definiscono relazioni tra ****chiavi primarie**** e ****chiavi esterne (foreign key)**** di tabelle diverse.
- * Servono a mantenere la ****consistenza dei dati tra più tabelle****.

Esempio:

Abbiamo due tabelle:

```
CLIENTE(IdCliente PRIMARY KEY, Nome, Cognome)
ORDINE(IdOrdine PRIMARY KEY, Data, IdCliente FOREIGN KEY REFERENCES
CLIENTE(IdCliente))
```

* Il vincolo referenziale assicura che in `ORDINE.IdCliente` possano comparire **solo valori esistenti** in `CLIENTE.IdCliente`.

* Impedisce, ad esempio, di registrare un ordine per un cliente che non esiste nel database.

In SQL:

```
FOREIGN KEY (IdCliente) REFERENCES CLIENTE(IdCliente)
```

Opzioni di integrità referenziale:

* **ON DELETE CASCADE** → se un cliente viene eliminato, si eliminano anche i suoi ordini.

* **ON DELETE SET NULL** → se un cliente viene eliminato, negli ordini il campo `IdCliente` diventa `NULL`.

* **ON DELETE RESTRICT / NO ACTION** → impedisce l'eliminazione di un cliente se ha ordini associati.

Da non fare: permettere ordini senza cliente (chiave esterna facoltativa)

Si rende **nullable** la colonna IdCliente:

```
CREATE TABLE Ordine (
  IdOrdine INT PRIMARY KEY,
  Data DATE,
  IdCliente INT NULL, -- può essere vuoto
  FOREIGN KEY (IdCliente) REFERENCES Cliente(IdCliente)
);
```

➡ In questo modo:

- puoi inserire IdCliente = NULL → ordine “anonimo”
- ma se metti un valore, deve comunque esistere in Cliente.

In sintesi

* I **vincoli di dominio** controllano **i valori ammessi in una singola colonna**.

* I **vincoli referenziali** controllano la **coerenza tra più tabelle collegate**.

6. Conclusione

- **Modello concettuale** → pensare ai dati in modo astratto.

- **Modello logico** → trasformare quel pensiero in relazioni (tabelle).
- Questi passaggi sono fondamentali per costruire un database ben progettato, evitando ridondanze e incoerenze.

Traccia della prova di maturità informatica

Una traccia “standard” di una prova della maturità di informatica richiede:

1. Analisi della realtà descritta: comprendere i soggetti, i dati, le relazioni, i vincoli impliciti.
2. Schema concettuale (diagramma E-R): entità, attributi, relazioni, cardinalità, chiavi, eventuali vincoli particolari.
3. Schema logico (relazionale): traduzione in tabelle, con chiavi primarie, chiavi esterne, colonne, tipi dei dati, vincoli di integrità.
4. Query SQL su alcune tabelle.
5. Spesso anche la parte web: interfacce, pagine da progettare, segmenti di codice o pseudocodice che interagiscono col database.

Esempio concreto (pratico) – Car pooling

Prendiamo come esempio la **prova del 2017** per “Informatica e telecomunicazioni, articolazione Informatica”, perché è ben documentata. [Zanichelli](#) | [Zanichelli](#)

1. Realtà descritta

Nel testo:

- C'è una piattaforma web per **car pooling** nazionale. [Zanichelli](#) | [Zanichelli](#)
- Tipologie di utenti: **autisti** (offrono passaggi) e **passaggeri** (usufruiscono). [Zanichelli](#) | [Zanichelli](#)
- Autisti inseriscono dati personali, patente, auto, contatti, foto. [Zanichelli](#) | [Zanichelli](#)
- Ogni viaggio (offerto da autisti) ha: città di partenza, destinazione, data/ora, contributo richiesto, tempi stimati. [Zanichelli](#) | [Zanichelli](#)

Altre specifiche: può esserci la registrazione degli utenti, magari gestione dei passaggi, prenotazioni, oppure feedback/commenti, ecc.

2. Modello concettuale (schema E-R)

Da questa descrizione si costruisce un modello concettuale:

- **Entità** principali:
 - **Utente** (oppure potresti dividere in Autista e Passeggero, a seconda se hanno attributi molto diversi)
 - **Auto** (se autisti hanno auto proprie, che possono usare per offrire viaggi)
 - **Viaggio**

- **Attributi:**
 - Utente: idUtente (chiave primaria), nome, cognome, email, telefono, data di registrazione, fotografia, ecc.
 - Autista (se distinto): numero patente, scadenza patente.
 - Auto: marca, modello, targa, capienza posti, anno, ecc.
 - Viaggio: idViaggio, cittàPartenza, cittàArrivo, data, ora, contributo richiesto, tempo stimato, idAutista (chiave esterna), ecc.
- **Relazioni:**
 - Un autista “offre” **Viaggi** → Relazione 1:N da Autista a Viaggio (un autista può offrire molti viaggi, ogni viaggio è offerto da un solo autista).
 - Un passeggero può prenotare un Viaggio → Relazione N:M tra Utente (Passeggero) e Viaggio, con magari una entità relazione “Prenotazione” che ha data di prenotazione, numero di posti prenotati, stato della prenotazione, ecc.
- **Vincoli di cardinalità:**
 - Un utente autista può essere anche passeggero? Dipende dal testo, se è ammesso (se si permette che lo stesso utente faccia entrambe le funzioni).
 - Ogni viaggio deve avere almeno un autista, etc.

3. Modello logico (schema relazionale)

Dal modello concettuale, si definisce il modello logico, con le tabelle:

Esempio di schema logico:

Tabella	Colonne principali	Chiave primaria	Chiavi esterne
Utente	idUtente, nome, cognome, email, telefono, foto, tipoUtente (autista/passeggero)	idUtente	—
Auto	idAuto, marca, modello, targa, posti, anno, idUtente (autista proprietario)	idAuto	idUtente → Utente
Viaggio	idViaggio, cittàPartenza, cittàArrivo, data, ora, contributo, tempoStimato, idAuto	idViaggio	idAuto → Auto
Prenotazione	idPrenotazione, idViaggio, idUtente (passeggero), dataPrenotazione, numeroPosti, stato	idPrenotazione	idViaggio → Viaggio, idUtente → Utente

- Si definisce il tipo dei campi (es. idUtente: integer, nome: varchar(...), data: date etc.).
- Si indicano obbligatorietà / opzionalità (es. data/ora di partenza obbligatoria, magari foto facoltativa).
- Si stabiliscono i vincoli referenziali: ogni prenotazione deve riferirsi a un viaggio esistente; ogni viaggio deve riferirsi a una auto/autista esistente; etc.

Come si lega al “modello concettuale e logico” richiesto dalla prova

- Il **modello concettuale** è la parte in cui disegni il diagramma E-R, identifichi le entità, le relazioni, le cardinalità, le chiavi, gli attributi. Serve a capire bene cosa rappresentare.

- Il **modello logico** è la traduzione in tabelle, definizione di chiavi, chiavi esterne, tipi, eventuali vincoli, domini dei dati. Serve ad avere una struttura che può essere implementata in SQL.